

Jake DeMuesy

CS 499

6/28/26

Category Three – Narrative

Narratives: Provide a narrative to accompany each artifact in your ePortfolio. The narrative that you write for each artifact should explain why you included the artifact in your ePortfolio and reflect on the process you used to create the artifact. The narrative should focus less on the actual creation of the artifact and more on the learning that happened through the creation of the artifact. Address the following items for each artifact:

Briefly **describe** the artifact. What is it? When was it created?

This artifact is the Grazioso Salvare Animal Rescue Dashboard originally built in CS 340, Client/Server Development, probably in late 2025. The premise is a full-stack web application for a fictional animal rescue organization that uses Austin Animal Center outcome data. The original version was a Jupyter Dash notebook with a PyMongo CRUD module, a data table, a breed distribution pie chart, and a Leaflet geolocation map. Users could filter the table by rescue type (water, mountain/wilderness, disaster/individual tracking) and click a row to drop a pin on the map for that animal's recorded location.

The enhanced version kept that core structure but added a meaningful amount of backend engineering on top of it. The key additions were: role-based access control in the CRUD module (admin vs analyst roles with write-gate enforcement), compound and single-field indexes on the filter query fields, a server-side aggregation layer extracted into its own module called `animal_analytics.py`, a shared query builder function so the table and aggregation chart can't drift apart, and a test suite for the analytics module that runs without a live MongoDB connection.

Justify the inclusion of the artifact in your ePortfolio. Why did you select this item? What specific components of the artifact showcase your skills and abilities in software development? How did the enhancement improve the artifact? What specific skills did you demonstrate in the enhancement?

I picked this artifact because it covers database engineering in a context that actually looks like a real application -- not just raw query exercises but a working UI that connects to a live MongoDB instance with real data going through it. The enhancements I made address security, performance, and architecture at the same time, which gave me a lot to write about and defend.

The RBAC implementation is probably the most directly demonstrable security work in the portfolio. The CRUD module now enforces two roles at the application level: analyst (read-only, the default) and admin (required for create, update, delete). The `_check_write()` method gates every write operation and raises a `PermissionError` if the connected role isn't admin. The dashboard itself connects as analyst, so the production path literally cannot write to the database by construction. There's also a comment block at the bottom of the CRUD module showing the corresponding mongosh commands to create the database-level users with the right permissions (readWrite for admin, read for analyst), so the server-side and application-level access controls match.

The indexing work is the performance piece. The compound index covers `animal_type`, `sex_upon_outcome`, `breed`, and `age_upon_outcome_in_weeks` -- in that order -- which matches the exact field pattern the rescue-type filter queries use (three equality fields followed by a range field on age). That order is intentional; putting equality predicates before the range field lets MongoDB use the index efficiently instead of falling back to a collection scan. The `ensure_indexes()` method is safe to call every time the notebook starts because `create_index()` is a no-op if the index already exists. The `explain_query()` helper wraps MongoDB's `explain()` output so you can verify an IXSCAN is actually happening rather than a COLLSCAN -- that's empirical validation, not just assumption.

Separating `animal_analytics.py` from the CRUD module is the architecture piece. The original dashboard did all its counting and grouping in Python after pulling records from the database -- the breed pie chart was just `pandas.value_counts()` on whatever rows were in the table. The enhanced version pushes that work into MongoDB's aggregation pipeline. `by_breed()`, `by_outcome()`, and `over_time()` each build a pipeline that runs `$group`, `$sort`, and `$project` server-side, so only the summarized result comes back over the wire instead of every matching document. The `by_outcome()` result feeds the new bar chart in the dashboard through a dedicated Dash callback, using `analytics.by_outcome()` with the same query that the table filter uses -- both built by the same `build_rescue_query()` helper so they can't diverge.

The test suite added in the final round of feedback is `test_animal_analytics.py`. It validates all three aggregation methods using a `StubCollection` that simulates MongoDB's `aggregate()` pipeline in Python, so the tests run without any live database connection. Eight tests cover correct

counts, top_n limiting, query filtering, granularity switching in over_time(), and empty collection handling. All eight pass against the existing animal_analytics.py without touching it.

Reflect on the process of enhancing the artifact. What did you learn as you were creating it and improving it? What challenges did you face? How did you incorporate feedback as you made changes to the artifact? How was the artifact improved? Which course outcomes did you partially or fully meet with your enhancements? Which do you feel were not met?

The thing I thought about most on this enhancement was where to draw the line between the CRUD module and the analytics module. At first it seemed like the aggregation methods could just be extra methods on AnimalShelter -- they still use the same collection, same connection. But the more I looked at it, the more those two layers are doing different things. AnimalShelter manages the connection lifecycle, enforces access control, and handles document-level operations. AnimalAnalytics builds pipelines and interprets results for reporting purposes. If you put both in the same class, a change to the aggregation logic risks touching the CRUD logic and vice versa -- and since AnimalShelter is the thing the dashboard depends on for every read and write, you don't want it changing for reporting reasons. Keeping them separate is a real trade-off, not just a style preference.

The RBAC implementation was pretty straightforward to build but it made me think about where access control actually belongs in a layered system. Enforcing the analyst role at the application layer in _check_write() is useful for catching mistakes -- if a developer accidentally calls create() with an analyst connection, they get a clear PermissionError instead of a vague MongoDB authentication error. But it's not a replacement for database-level user permissions. The real security is the mongosh user configuration that gives aacanalyst only the read role on the aac database. The application-level check is defense in depth, not the primary gate. Getting both right -- and understanding why both matter -- is the part of this that felt most like something I'd actually defend in an interview.

The aggregation pipeline work had a specific technical wrinkle in over_time() that was worth figuring out. The Austin Animal Center dataset stores the datetime field as a string like '2015-12-28T08:00:00' rather than a BSON date type. That means you can't use MongoDB's \$dateToString or \$dateTrunc operators directly -- they expect actual date types. The workaround is \$substr to pull the first 4 characters for year grouping or first 7 for year-month, which works fine for string dates in ISO format. It's a little inelegant but it's correct and it's honest about why it's done that way, which is what the comment in the code says.

Incorporating the feedback from Dr. Bolton was pretty easy on this one because his response was more validation than correction -- he confirmed the separation of concerns, the server-side aggregation, and the RBAC/indexing work were all on the right track. The only forward-looking note was about testing analytical execution paths, which is what the test suite

addresses. Writing `test_animal_analytics.py` required building a stub collection that simulates MongoDB's `aggregate()` behavior in Python, which turned out to be more involved than expected -- each pipeline stage has to be implemented separately (`$match`, `$group`, `$sort`, `$limit`, `$project`) and the `$group` stage in particular has to handle both simple field references and the `$substr` expression used in `over_time()`. The upside is that the tests now document the expected behavior of each method clearly enough that someone could use them to understand the API without reading the implementation.

In terms of course outcomes, this enhancement covers the security outcome most directly -- RBAC and least-privilege access are exactly what that outcome is asking for. The database outcome is covered through the aggregation pipeline work and compound indexing. The software engineering outcome runs through the separation of concerns between CRUD and analytics, and the test suite. The professional communication outcome is in the narrative and the code comments explaining design decisions. The algorithms outcome was handled more directly in Enhancement Two, so I'd say this enhancement is strongest on security and databases, which is appropriate for what the artifact is.